

Linked Lists

A pattern-recognition study guide: pointer manipulation, fast/slow, reversal, merging, and the classic interview problems

Derived from a Socratic-style study session · every solution tested against hundreds of random inputs

Roadmap

This guide follows one core idea: **a linked list problem is almost never a brand-new algorithm — it's a combination of a small number of pointer-manipulation patterns.** Once you can recognize which pattern (or which *composition* of patterns) a problem wording is pointing at, the code mostly writes itself.

The five rules that everything else is built from:

1. Before changing a link, ask: "*Do I still need the old `.next` ?*"
2. If yes, **save it first** — a local variable is cheap; a lost reference is not.
3. If deleting a node, think about its **predecessor** — you always need the node *before* the one you're removing.
4. If the head itself needs to change, consider a **dummy node** — it manufactures a predecessor for the head so no code path is a special case.
5. If the problem involves positions relative to **both ends** of the list (middle, kth-from-end, cycle), think **two pointers** moving at different speeds or with a fixed gap.

Part 1 — Fundamentals. Node anatomy, the crucial distinction between moving a pointer and changing the list, traversal, deletion, and why the dummy node exists.

Part 2 — Reversal. The save → reverse → advance loop, derived step by step, with the reversal invariant that generalizes to every variant (k-groups, reverse-between, ...).

Part 3 — Fast & Slow Pointers. Finding the middle (both conventions), Floyd's cycle detection, the cycle-entry derivation (including a real correction made mid-derivation), and kth-from-end via a fixed gap.

Part 4 — Deletion & Merge Patterns. Removing the Nth node from the end, merging two sorted lists, and the composite patterns built from reversal + fast/slow: reorder list and palindrome check.

Part 5 — Cross-Structure Patterns. Intersection of two lists, doubly linked lists, and the LRU cache (hashmap + DLL combination).

Part 6 — Advanced Patterns. Reverse nodes in k-groups, merge k sorted lists (min-heap), and sorting a linked list (merge sort).

Part 7 — More Classics. Copy list with random pointer, add two numbers, and rotate list.

Part 8 — Extension Problems. Partition list, remove duplicates II, swap pairs, reverse between, odd-even list, and flatten a multilevel doubly linked list — problems not solved in the source material but built from the exact same toolkit.

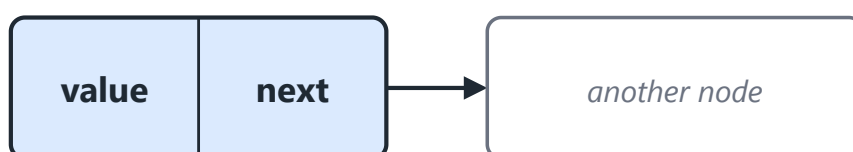
Part 9 — Recognition, Templates & Interview Q&A. The master pattern-recognition table, condensed code templates, a decision guide, and a Socratic-style review.

Part 1 — Fundamentals

1.1 The mental model

Forget arrays for a moment. A linked list is a chain of **nodes**, and each node holds two things: a value, and a reference to the next node.

Anatomy of a node



next does not hold the next node's VALUE.

next holds a REFERENCE to the next node object itself.

The critical detail: `next` does **not** store the next node's value. It stores a *reference to the node object itself*. `head` is nothing special either — it's just a reference (a variable) pointing at the first node. If `head = None`, the list is empty. A node whose `next` is `None` is the last node.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

1.2 Why not just use an array?

An array gives $O(1)$ random access — `arr[3]` jumps straight to the fourth element because the memory layout is contiguous and indexable. A linked list has no such shortcut: to reach the 4th node you must walk through the first three, so random access is $O(n)$.

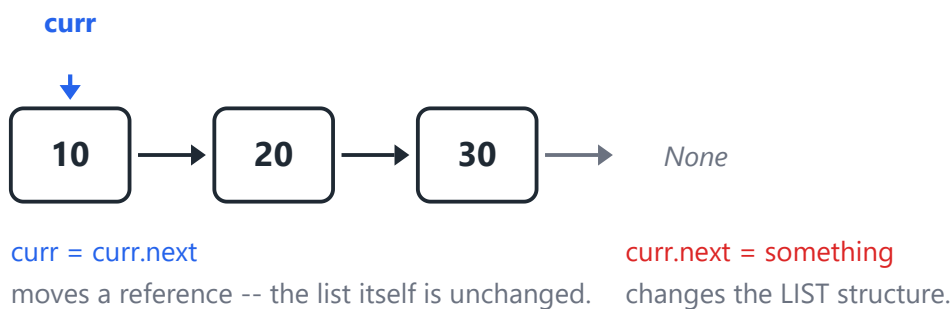
What a linked list buys you instead is **cheap structural change**: inserting a node between two existing nodes means rewiring two references, not shifting every element after the insertion point the way an array would. But be careful — insertion/deletion isn't *automatically* $O(1)$. You need a reference to the relevant node already in hand; *finding* that position can still cost $O(n)$.

Trade-off to remember: linked lists trade fast access for potentially cheap structural changes — "potentially" because you still need the position.

1.3 Moving a pointer vs. changing the list

This is the single most important distinction in the entire topic, and the one nearly every bug traces back to.

Moving a pointer vs. changing the list



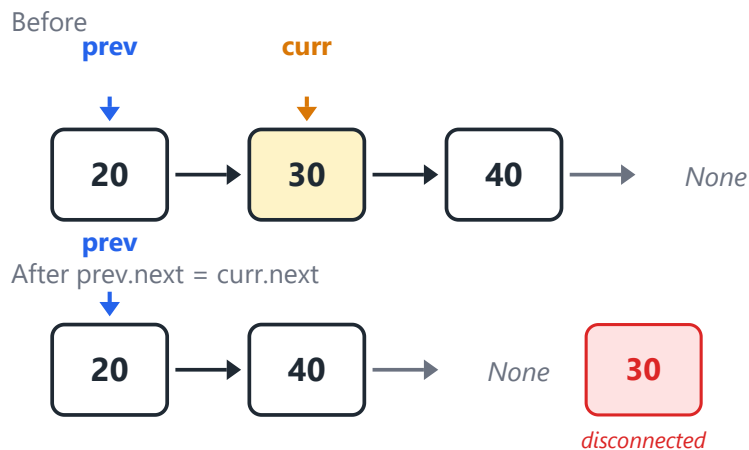
- `curr = curr.next` **moves a reference.** No node's `.next` field changed. The list itself is completely untouched — only your view into it moved.
- `curr.next = something` **rewrites the list's structure.** This is the only kind of statement that actually changes what the list looks like.

Internalizing this distinction early is what makes reversal, deletion, and every pointer-heavy problem in this guide tractable instead of confusing.

1.4 Deleting a node

Suppose we want to delete a node given `prev` (the node just before it) and `curr` (the node to delete):

Deleting a node: `prev.next = curr.next`



```
prev.next = curr.next
```

`curr` becomes unreachable from `head` — that is deletion in a linked list. You don't destroy the node object; you just remove the chain's only path to it. (In Python it may briefly still exist if a variable still references it, but nothing reachable from `head` can get to it anymore.)

The boundary case: what is `prev` when you want to delete the **head**? There isn't one. `prev.next = curr.next` simply doesn't apply. For the head specifically, you instead do:

```
head = head.next
```

This asymmetry — "ordinary nodes have a predecessor, the head doesn't" — is exactly the problem the dummy node exists to solve.

1.5 The dummy node

Deleting the head needs a predecessor -- use a dummy

Without dummy: deleting head is a special case



A dummy node is a throwaway node placed *before* the real head:

```
dummy = ListNode(0)
dummy.next = head
```

Now **every** node, including the original head, has a predecessor (`dummy` is the head's predecessor). The exact same `prev.next = curr.next` operation now works uniformly whether `curr` is the original head or somewhere in the middle — no special case needed. At the end, you return `dummy.next` (never `dummy` itself — it was never part of the real list).

This single trick removes almost all of the "what if it's the first node?" edge-case handling you'd otherwise need across deletion, merging, and reordering problems.

```
def build_list(values) -> Optional[ListNode]:
    dummy = ListNode(0)
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
```

```
def to_list(head) -> list:
    out = []
    seen = set()
    while head:
        if id(head) in seen: # safety: never infinite-loop on a stray cycle
            out.append("...CYCLE...")
            break
        seen.add(id(head))
        out.append(head.val)
        head = head.next
    return out
```

(`build_list` / `to_list` are test helpers used throughout this guide and `test_sol.py` to convert between Python lists and `ListNode` chains — they aren't LeetCode-graded code, just plumbing.)

Part 2 — Reversal

2.1 Why reversal is dangerous

Reversing `1 -> 2 -> 3 -> 4` into `4 -> 3 -> 2 -> 1` means, at some point, executing something like `curr.next = prev`. But the moment you do that, you've **overwritten your only path forward** — whatever `curr.next` used to point to is gone unless you saved it first.

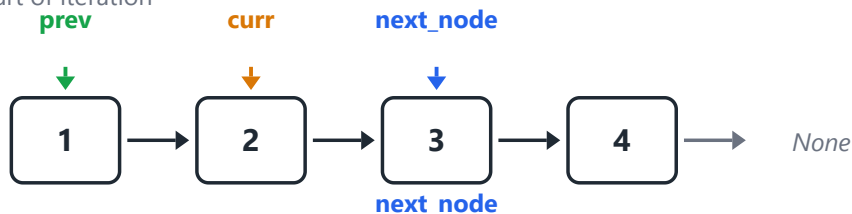
Rule: before changing a link, ask "am I destroying my only path to something I still need?" If yes, save a reference to it *first*.

2.2 Deriving the loop, one step at a time

Start with three pointers: `prev` (the already-reversed portion, initially `None`), `curr` (the first unprocessed node), and a temporary `next_node`.

One reversal iteration: save -> reverse -> advance

Start of iteration



After `curr.next = prev` (the `2→1` link is now reversed)



`prev = curr; curr = next_node` comes next, repeating the invariant.

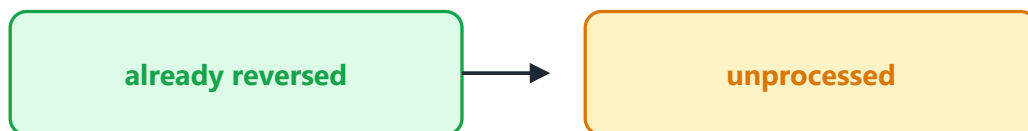
One iteration, in the order that matters:

```
next_node = curr.next    # 1. Save – we're about to overwrite curr.next
curr.next = prev         # 2. Reverse – point curr backward
prev = curr              # 3. Advance prev to the node we just reversed
curr = next_node         # 4. Advance curr to the saved next node
```

Trace it by hand once: starting from `prev -> None`, `curr -> 1 -> 2 -> 3 -> 4`, after one iteration `prev -> 1 -> None` and `curr -> 2 -> 3 -> 4`. The list `1 -> 2 -> ...` didn't get shorter or corrupted — node `1` just now points backward at `None` instead of forward at `2`, and both `prev` and `curr` advanced by exactly one node.

2.3 The invariant

Reversal invariant



`curr` always points to the first unprocessed node.

At every point in the loop:

Everything before `curr` has already been reversed. `curr` is the first node that has not yet been processed.

This is the mental model to hold onto — not the four lines of code. If you can state the invariant, you can re-derive the loop for any list at any point, including mid-list variants like "reverse only positions `[left, right]`" (Part 8).

2.4 When does the loop stop, and what do we return?

The loop runs `while curr:`. It stops exactly when `curr` becomes `None` — i.e. when there's nothing left to process. At that moment, by the invariant, **everything** has been reversed, and `prev` is sitting on the last node processed, which is the new head.

```
def reverseList(head: Optional[ListNode]) -> Optional[ListNode]:
    prev = None
    curr = head

    while curr:
        next_node = curr.next    # Save
        curr.next = prev        # Reverse
        prev = curr              # Advance prev
        curr = next_node        # Advance curr

    return prev
```

Complexity: $O(n)$ time (every node visited once), $O(1)$ extra space (just three pointers).

2.5 Trace

iteration	prev chain	curr
1	1	2
2	2 -> 1	3
3	3 -> 2 -> 1	4
4	4 -> 3 -> 2 -> 1	None

Final reversed list: 4 -> 3 -> 2 -> 1

2.6 Recognition

Whenever a problem says *reverse the list*, *reverse a portion*, *reverse the first k* , or *reverse in groups of k* — think:

prev + curr + a saved next_node , applied to whichever window of the list the problem is asking about.

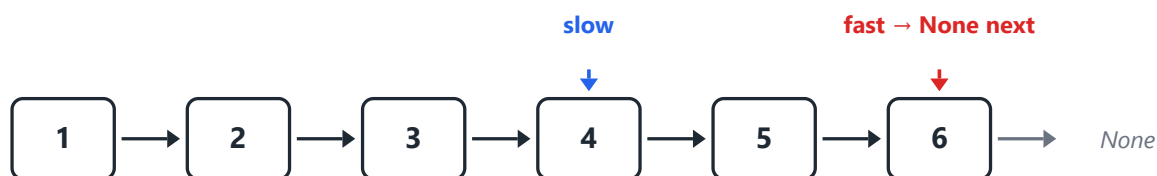
You'll see this exact loop reused, unmodified, inside reorder list, palindrome check, sort-by-merge's building blocks, reverse-k-group, and reverse-between later in this guide.

Part 3 — Fast & Slow Pointers

3.1 Deriving why it works

Rather than memorizing "slow moves 1, fast moves 2," derive it: start both pointers at `head`. Every round, `slow` advances 1 node and `fast` advances 2. Since `fast` covers exactly twice the distance of `slow` in the same number of rounds, **by the time `fast` has traveled the whole list, `slow` has traveled roughly half of it.** That's the entire mechanism — no more, no less.

Fast/slow: fast covers 2x the distance of slow



while fast and fast.next: slow += 1, fast += 2

When fast falls off the end, slow sits at the middle (second middle for even length).

3.2 Two middles, two setups

For an **even-length** list there are two valid "middles." Which one you land on depends only on where `fast` starts and what the loop condition checks:

```
# second middle (fast starts at head)
slow = head
fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next

# first middle (fast starts one node ahead)
slow = head
fast = head.next
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
```

```
def middleNode(head: Optional[ListNode]) -> Optional[ListNode]:
    """Returns the SECOND middle for even-length lists (LC 876)."""
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow
```

```
def firstMiddle(head: Optional[ListNode]) -> Optional[ListNode]:
    """Returns the FIRST middle for even-length lists."""
    if not head:
        return None
    slow = head
    fast = head.next

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow
```

Why the guard is `fast and fast.next` , not just `fast` : if `fast` is on the last node, `fast.next` is `None` , and evaluating `fast.next.next` next would be `None.next` — a crash. Checking both first guarantees the two-step advance is always safe.

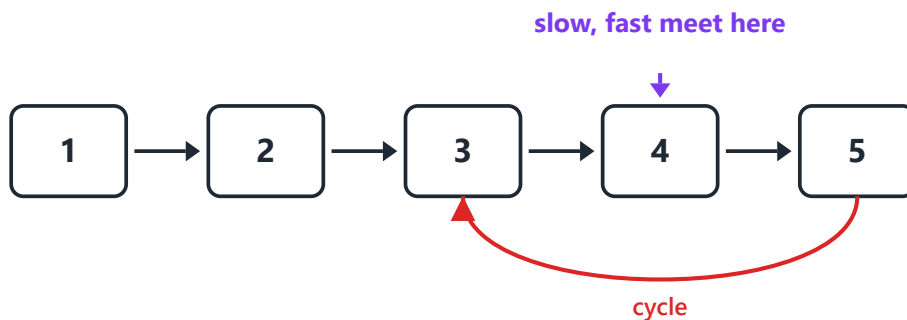
round	slow	fast
0	1	1
1	2	3
2	3	5
3	4	None

middleNode([1, 2, 3, 4, 5, 6]) = 4 (second middle)

Recognition: "middle of linked list" → `slow = 1x, fast = 2x` . The *starting position* of `fast` is what changes which middle you get — that single fact generalizes to a lot of off-by-one situations in fast/slow problems.

3.3 Cycle detection — Floyd's algorithm

Floyd's cycle detection



slow moves 1 step, fast moves 2. If a cycle exists, fast laps slow and they collide.
If fast reaches None first, there is no cycle.

Reframe the question: instead of "does traversal eventually reach `None`?", ask "will two pointers moving at different speeds ever land on the *same node*?" If a cycle exists, the faster pointer eventually laps the slower one inside the cycle and they collide. If no cycle exists, `fast` simply reaches the end first.

```
def hasCycle(head: Optional[ListNode]) -> bool:
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow is fast:
            return True

    return False
```

Recognition: "does the list contain a cycle?" → Floyd / fast + slow, checking `slow is fast`.

3.4 Cycle entry — and a real correction

Knowing *that* a cycle exists is easy. Finding *where it starts* is the harder, more interesting problem — and it's a good example of why you should never accept a claimed relationship without deriving it.

The claimed trick: once `slow` and `fast` meet somewhere inside the cycle, reset one pointer to `head`, then move both one step at a time. Where they meet again is the cycle's entry point.

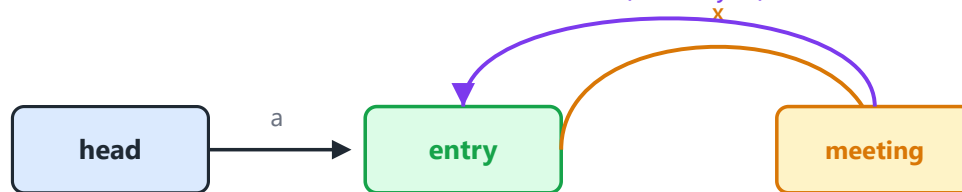
```
slow = head
while slow != fast:
    slow = slow.next
    fast = fast.next
return slow
```

The session first stated the justification as simply " $distance(head \rightarrow entry) = distance(meeting \rightarrow entry)$ " — and was immediately challenged: **where does that come from?** That challenge is worth preserving, because the first version turns out to be imprecise.

Correction, worked through live

Let a = distance from `head` to the cycle entry, L = the cycle's length, and x = distance from the entry to the meeting point (going forward around the cycle).

Cycle-entry distances: $\text{head} \rightarrow \text{entry} \equiv \text{meeting} \rightarrow \text{entry} \pmod{L}$



$$2S = a + x + kL \text{ and } S = a + x \Rightarrow a + x = kL \Rightarrow a \equiv -x \equiv L - x \pmod{L}$$

So resetting one pointer to `head` and walking both 1 step at a time meets at entry.

At the moment `slow` and `fast` first meet, `slow` has traveled $s = a + x$ steps (it reaches the meeting point without necessarily completing a full lap). `fast` moves twice as fast, so it has traveled $2s$, but its path is `head` $\rightarrow$ `entry` $\rightarrow$ *some whole number of laps* $\rightarrow$ `meeting`, i.e. $2s = a + x + kL$ for some integer $k \geq 0$.

Substituting $s = a + x$:

$$\begin{aligned} 2(a + x) &= a + x + kL \\ a + x &= kL \\ a &= kL - x \end{aligned}$$

By definition of modular congruence, $a = kL - x$ means $a \equiv -x \pmod{L}$. Since adding a multiple of L doesn't change a value mod L , $-x \equiv L - x \pmod{L}$, so:

$$a \equiv L - x \pmod{L}$$

And $L - x$ is exactly the remaining distance around the cycle **from the meeting point back to the entry**. So the precise statement is not "the two distances are equal" — it's:

distance(head $\rightarrow$ entry) $\equiv$ distance(meeting $\rightarrow$ entry) (mod the cycle length L).

That's why resetting one pointer to `head` and advancing both one step at a time makes them arrive at the entry together: they're walking congruent distances around a structure that repeats every `L` steps, so being off by a whole number of laps doesn't matter.

```
def detectCycle(head: Optional[ListNode]) -> Optional[ListNode]:
    slow = head
    fast = head

    # Phase 1: detect cycle
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow is fast:
            break
    else:
        return None

    # Phase 2: find entry
    slow = head

    while slow is not fast:
        slow = slow.next
        fast = fast.next

    return slow
```

phase	round	slow	fast
1	1	2	3
1	2	3	5
1	3	4	4
2	0	1	4
2	1	2	5
2	2	3	3

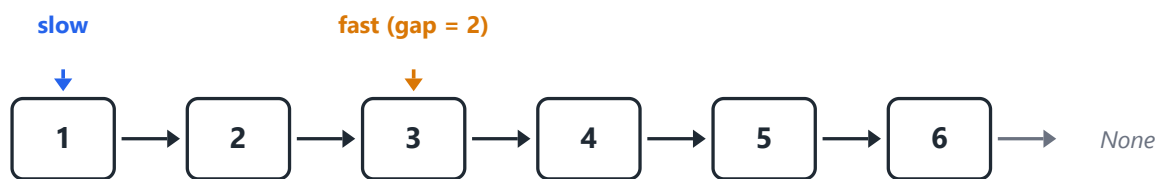
Cycle entry found at node with value 3 (meeting point was 4)

Recognition: "find where the cycle starts" → Floyd phase 1 (find a meeting point) → reset one pointer to `head` → phase 2 (walk both one step at a time until they meet again).

3.5 Kth node from end — fixed gap, not length

A natural but clumsy approach is to first walk the list to find its length, then walk again to the target. The fast/slow toolkit does it in one pass: create a **fixed gap of `k`** between two pointers, then advance both together. When the leading pointer falls off the end, the trailing one is sitting exactly `k` nodes from the end.

Kth from end: fixed gap of `k`



Advance fast by `k` first, then move both together; when fast hits `None`, slow is `k`-th from the end.

```
def kthFromEnd(head: Optional[ListNode], k: int) -> Optional[ListNode]:
    fast = head

    for _ in range(k):
        fast = fast.next

    slow = head

    while fast:
        slow = slow.next
        fast = fast.next

    return slow
```

Recognition: "kth from the end" → create a gap of `k`, then move both together.

Part 4 — Deletion & Merge Patterns

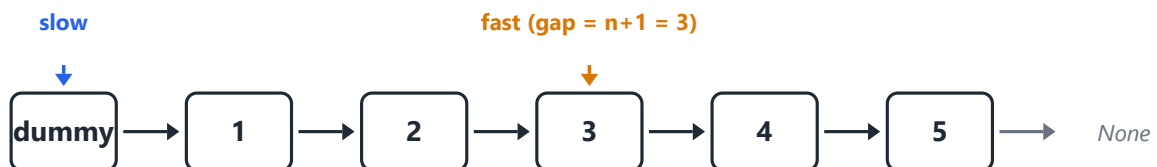
4.1 Remove Nth node from end — dummy + gap (k+1)

Section 3.5's fixed-gap trick finds the target node itself. But deleting a node needs its **predecessor**, not the node. So the gap changes:

```
Find kth from end:   gap = k       -> slow lands ON the target
Delete kth from end: gap = k+1     -> slow lands BEFORE the target
```

Combine that `+1` with a dummy node (Section 1.5) so that even deleting the original head is an ordinary case:

Remove Nth from end: dummy + gap (n+1)



slow lands on the PREDECESSOR of the target, so `slow.next = slow.next.next` deletes it (`n=2`).

```
def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
    dummy = ListNode(0)
    dummy.next = head

    slow = dummy
    fast = dummy

    for _ in range(n + 1):
        fast = fast.next

    while fast:
        slow = slow.next
        fast = fast.next

    slow.next = slow.next.next

    return dummy.next
```

stage	slow	fast
advance fast (1/3)	0	1
advance fast (2/3)	0	2
advance fast (3/3)	0	3
advance both	1	4
advance both	2	5
advance both	3	None

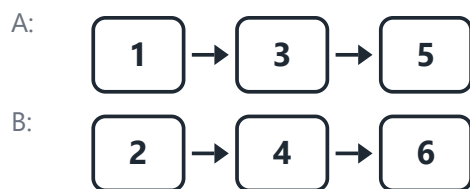
Remove 2nd-from-end of [1, 2, 3, 4, 5] -> [1, 2, 3, 5]

Recognition: "remove kth from end" → dummy + two pointers + gap $k+1$.

4.2 Merge two sorted lists

The mental model is the merge step of merge sort: two sorted streams, and at every step you compare their current heads and attach the smaller one.

Merge two sorted lists



result (dummy -> ... -> tail):



Compare the two current nodes, attach the smaller one, advance that list and tail.

The trick that removes all the "is this the first node?" bookkeeping is, again, a dummy node plus a **tail** pointer that always points at the last node already placed:

```
def mergeTwoLists(list1: Optional[ListNode], list2: Optional[ListNode]) ->
Optional[ListNode]:
    dummy = ListNode(0)
    tail = dummy

    while list1 and list2:
        if list1.val <= list2.val:
            tail.next = list1
            list1 = list1.next
        else:
            tail.next = list2
            list2 = list2.next

        tail = tail.next

    tail.next = list1 or list2

    return dummy.next
```

Invariant: everything from `dummy.next` through `tail` is already a correctly merged, sorted list; `list1` / `list2` are whatever hasn't been placed yet. Once one list runs out, the other is *already* sorted and already larger than everything placed so far — so `tail.next = list1 or list2` finishes the job with no more comparisons needed.

Complexity: $O(m + n)$ time, $O(1)$ extra space (existing nodes are reused, not copied).

chosen	remaining A	remaining B
1	3 5	2 4 6
2	3 5	4 6
3	5	4 6
4	5	6
5	-	6

Merged: 1 -> 2 -> 3 -> 4 -> 5 -> 6

Recognition: "merge sorted linked lists" → two current pointers + dummy + tail.

4.3 Reorder list — a composition, not a new algorithm

`1 -> 2 -> 3 -> 4 -> 5` should become `1 -> 5 -> 2 -> 4 -> 3`. This *looks* like a brand-new problem, but it's actually three patterns you already have, chained together:

Reorder list: middle -> reverse second half -> interleave

1. Split at middle



2. Reverse second half



3. Interleave: first, second, first, second ...



Save `first_next/second_next` before rewriting `.next` -- same rule as reversal.

1. **Find the middle** (fast/slow) — split into a first half and second half.
2. **Reverse the second half** (Part 2's exact loop).
3. **Interleave** the two halves, one node at a time.

The one new subtlety: when interleaving, you cannot blindly do `first.next = second;` `second.next = first.next` — by the time the second line runs, `first.next` has already been overwritten. Same rule as reversal: **save the link before you overwrite it.**

```

def reorderList(head: Optional[ListNode]) -> None:
    if not head or not head.next:
        return

    # 1. Find middle
    slow = head
    fast = head

    while fast.next and fast.next.next:
        slow = slow.next
        fast = fast.next.next

    # 2. Reverse second half
    second = slow.next
    slow.next = None

    prev = None
    curr = second

    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    second = prev

    # 3. Merge/interleave
    first = head

    while second:
        first_next = first.next
        second_next = second.next

        first.next = second
        second.next = first_next

        first = first_next
        second = second_next

```

stage	list
start	1 -> 2 -> 3 -> 4 -> 5
after find-middle + reverse + interleave	1 -> 5 -> 2 -> 4 -> 3

`reorderList([1, 2, 3, 4, 5]) -> [1, 5, 2, 4, 3]`

Recognition: "alternate front, back, front, back..." → middle + reverse + merge.

4.4 Palindrome linked list

Same decomposition, different final step:

Palindrome check: middle -> reverse -> compare

1 -> 2 -> 3 -> 2 -> 1



left = 1,2 right (reversed 2nd half) = 1,2 -- compare pairwise, all equal.

Odd-length middle node is skipped; it never needs to match anything.

1. Find the middle.
2. Reverse the second half.
3. **Compare** corresponding nodes from the front and the reversed back half, instead of interleaving them.

For an odd-length list, the true middle node never needs a partner, so it's simply skipped over.

```

def isPalindrome(head: Optional[ListNode]) -> bool:
    if not head or not head.next:
        return True

    # Find middle
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    # For odd length, skip middle
    if fast:
        slow = slow.next

    # Reverse second half
    prev = None
    curr = slow

    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    # Compare
    left = head
    right = prev

    result = True
    while right:
        if left.val != right.val:
            result = False
            break
        left = left.next
        right = right.next

    return result

```

Complexity: $O(n)$ time, $O(1)$ extra space — no copy of the list is made; the existing nodes are reused for the comparison (and, if you care about preserving the original structure, could be reversed back afterward).

Recognition: "is this list a palindrome?" → middle → reverse → compare.

Part 5 — Cross-Structure Patterns

5.1 Intersection of two linked lists

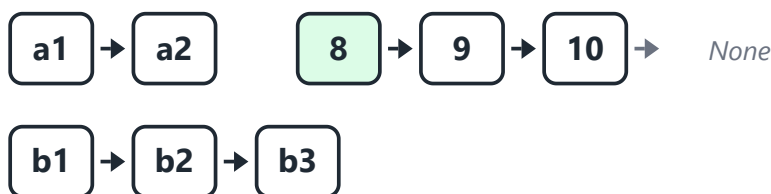
Two lists that share a common tail — meaning the *same node objects*, not merely equal values — intersect at the first shared node:

Intersection: swap heads to equalize total distance

A: 1 -> 2 -\

8 -> 9 -> 10

B: 3 -> 4 -> 5 -/



p travels A-only + shared + B-only; q travels B-only + shared + A-only -- equal totals.

p = p.next if p else headB / q = q.next if q else headA

The naive approach of walking both lists in lockstep fails whenever the lists have different lengths *before* the shared portion: the pointers would never land on the shared node at the same time. The fix is an elegant equalizing trick — when a pointer reaches the end of its own list, redirect it to the head of the *other* list:

```
def getIntersectionNode(headA: Optional[ListNode], headB: Optional[ListNode]) ->
Optional[ListNode]:
    if not headA or not headB:
        return None

    p = headA
    q = headB

    while p is not q:
        p = p.next if p else headB
        q = q.next if q else headA

    return p
```

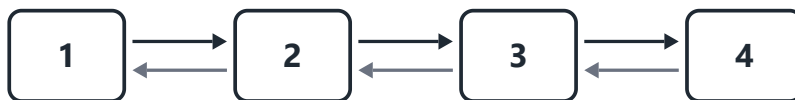
Why it works: let A = the length of A's unique prefix, B = the length of B's unique prefix, C = the shared length. Pointer p travels $A + C + B$ total steps (its own list, then the other's prefix); pointer q travels $B + C + A$. Both totals are identical, so if an intersection exists, both pointers arrive at the shared portion at exactly the same step. If there's no intersection, both pointers become `None` at the same step (having each walked $A + B$ total), and the loop correctly returns `None`.

Recognition: "intersection of two linked lists" → swap heads when a pointer reaches the end, to equalize total distance traveled.

5.2 Doubly linked lists

Each node in a doubly linked list carries **two** links — `prev` and `next` — so traversal can move in either direction:

Doubly linked list: $A.next == B$ and $B.prev == A$



Removing node 3 only needs 3's own neighbors:
`node.prev.next = node.next ; node.next.prev = node.prev`

```
class DNode:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None
```

The payoff: if you're already holding a reference to a node, you can remove it in $O(1)$ without needing a separately-tracked predecessor, because the node already knows its own neighbors:

```
def dll_remove(node: DNode) -> None:
    node.prev.next = node.next
    node.next.prev = node.prev
```

```
def dll_insert_between(left: DNode, node: DNode, right: DNode) -> None:
    left.next = node
    node.prev = left

    node.next = right
    right.prev = node
```

The invariant that must never be broken: for every adjacent pair $A \leftrightarrow B$, both $A.next == B$ **and** $B.prev == A$ must hold. Updating only one side (e.g. `node.prev.next = node.next` without also fixing `node.next.prev`) leaves the list in a corrupted, direction-inconsistent state — a very easy bug to introduce and a very useful thing to check for while debugging.

Recognition: whenever a problem needs $O(1)$ removal of an *arbitrary, already-known* node, a doubly linked list is a strong candidate.

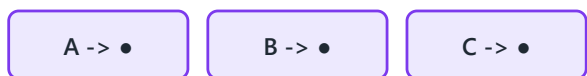
5.3 LRU Cache — HashMap + Doubly Linked List

This is the first problem in the guide that requires **combining two data structures**, because no single structure gives both operations the problem demands in $O(1)$:

- `get(key)` $\rightarrow O(1)$
- `put(key, value)` $\rightarrow O(1)$, evicting the least-recently-used entry when over capacity

LRU Cache: HashMap ($O(1)$ lookup) + Doubly Linked List ($O(1)$ reorder)

HashMap: `key -> node`



Doubly linked list (LRU on the left, MRU on the right):



`get/put` move the touched node to just before tail (MRU); eviction removes `head.next` (LRU).

A hash map alone gives $O(1)$ lookup but has no notion of "which key was used least recently." A linked list alone gives an ordering ($LRU \leftrightarrow \dots \leftrightarrow MRU$) but finding a node by key would take $O(n)$. Combine them:

```
HashMap:          key -> Node  
Doubly linked list:  LRU <-> ... <-> MRU
```

Two sentinel (dummy) nodes — `head` and `tail` — make every insertion/removal uniform, exactly as in Section 1.5, but for a doubly linked list on both ends at once.

```

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}

        self.head = DNode()
        self.tail = DNode()

        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node: DNode) -> None:
        prev = node.prev
        nxt = node.next

        prev.next = nxt
        nxt.prev = prev

    def _insert_mru(self, node: DNode) -> None:
        prev = self.tail.prev

        prev.next = node
        node.prev = prev

        node.next = self.tail
        self.tail.prev = node

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        node = self.cache[key]

        self._remove(node)
        self._insert_mru(node)

        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            node = self.cache[key]

            node.value = value
            self._remove(node)
            self._insert_mru(node)

            return

        node = DNode(key, value)

```

```
self.cache[key] = node
self._insert_mru(node)

if len(self.cache) > self.capacity:
    lru = self.head.next

    self._remove(lru)
    del self.cache[lru.key]
```

On get : find the node via the hashmap ($O(1)$), unlink it, and reinsert it right before `tail` (now the most-recently-used). **On put :** if the key exists, update its value and move it to MRU the same way; if it's new, insert at MRU and, if capacity is now exceeded, evict `head.next` (the actual LRU node) and drop it from the hashmap too.

Recognition: " $O(1)$ lookup **and** $O(1)$ ordered removal/reordering" $\rightarrow$ HashMap + Doubly Linked List. This combination shows up constantly in interviews beyond just LRU (LFU caches, "most recently used" browser history, etc.) — the pairing itself, not the specific problem, is what's worth memorizing.

Part 6 — Advanced Patterns

6.1 Reverse nodes in k-groups

1 -> 2 -> 3 -> 4 -> 5 -> 6 with $k = 3$ becomes 3 -> 2 -> 1 -> 6 -> 5 -> 4. If fewer than k nodes remain at the end, they're left untouched.

Reverse Nodes in k-Group ($k = 3$)

Before:



After:



Find the k -th node of each group, reverse just that group, then reconnect `group_prev` to the new group head.

You already know how to reverse a list (Part 2). The actual challenge here isn't the reversal itself — it's correctly locating where each group starts and ends, and reconnecting each reversed group back into the surrounding list. Track four reference points per group: `group_prev` (node before the group), `group_start` / `kth` (bounds of the group), and `group_next` (node after the group).

```

def reverseKGroup(head: Optional[ListNode], k: int) -> Optional[ListNode]:
    dummy = ListNode(0)
    dummy.next = head

    group_prev = dummy

    while True:
        # Find kth node
        kth = group_prev

        for _ in range(k):
            kth = kth.next

        if not kth:
            return dummy.next

        group_next = kth.next

        # Reverse group
        prev = group_next
        curr = group_prev.next

        while curr != group_next:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node

        # Reconnect
        old_start = group_prev.next
        group_prev.next = kth
        group_prev = old_start

```

group	reversed to
[1,2,3]	[3,2,1]
[4,5,6]	[6,5,4]

reverseKGroup([1, 2, 3, 4, 5, 6], k=3) -> [3, 2, 1, 6, 5, 4]

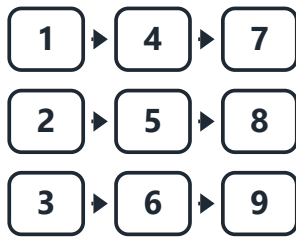
Recognition: "reverse every k nodes" → find group boundaries → reverse just that group
→ reconnect → repeat. Complexity: O(n) time, O(1) extra space.

6.2 Merge k sorted linked lists

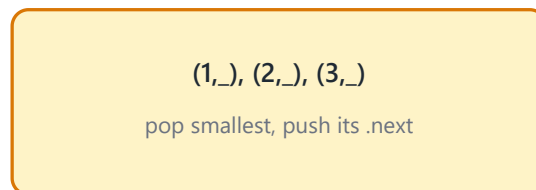
Repeatedly merging two lists at a time (`merge(l1, l2)` , then `merge(result, l3)` , ...) works but is wasteful. The better framing: at every moment, you need **the smallest current node among all k lists** — and a min-heap is exactly the structure built for "give me the smallest thing, repeatedly."

Merge K sorted lists with a min-heap

Lists:



heap holds each list's current head:



Every pop is $O(\log k)$; N total nodes give $O(N \log k)$ time, $O(k)$ heap space.

Tuple key (val, counter, node) breaks ties without ever comparing ListNode objects.

Seed the heap with the first node of every list. Then: pop the smallest, append it to the answer, and if the node it came from has a `.next` , push that in its place.

```
def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    heap = []
    counter = 0

    for node in lists:
        if node:
            heapq.heappush(heap, (node.val, counter, node))
            counter += 1

    dummy = ListNode(0)
    tail = dummy

    while heap:
        _, _, node = heapq.heappop(heap)

        tail.next = node
        tail = node

        if node.next:
            heapq.heappush(heap, (node.next.val, counter, node.next))
            counter += 1

    tail.next = None

    return dummy.next
```

Why the counter in the heap tuple? Python compares tuples element-by-element, and if two nodes have equal `.val`, it would fall through to comparing the `ListNode` objects themselves — which have no defined ordering and would raise a `TypeError`. A monotonically increasing counter as the second tuple element guarantees ties are broken without ever touching the node objects for comparison.

Complexity: for `k` lists totaling `N` nodes, $O(N \log k)$ time (`N` heap operations, each $O(\log k)$), $O(k)$ space for the heap.

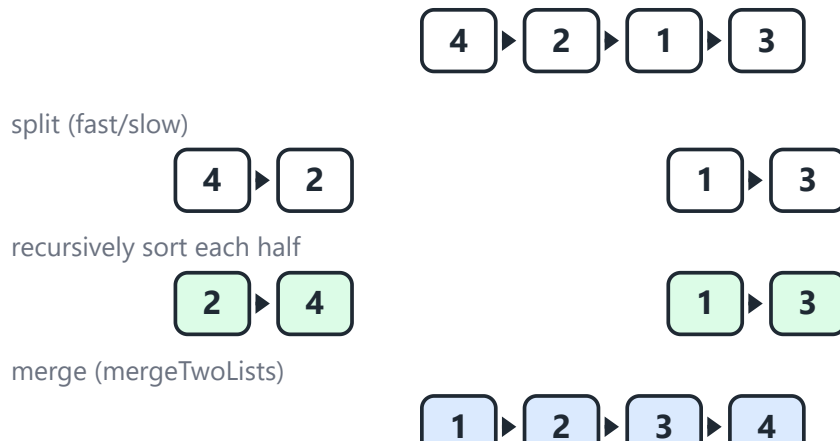
Recognition: many sorted streams where you repeatedly need the current smallest element → min-heap.

6.3 Sort a linked list

`4 -> 2 -> 1 -> 3` should become `1 -> 2 -> 3 -> 4`, ideally in $O(n \log n)$ time and $O(1)$ extra space beyond recursion.

Array-style sorts lean on random access, which linked lists don't have — but **merge sort** doesn't need random access at all: splitting is a single fast/slow pass, and merging is exactly Section 4.2's `mergeTwoLists` .

Sort a linked list: merge sort (split naturally, merge naturally)



find middle -> split into two halves -> sort each half recursively -> merge

```
def sortList(head: Optional[ListNode]) -> Optional[ListNode]:
    if not head or not head.next:
        return head

    # Find middle (first-middle split so both halves shrink each level)
    slow = head
    fast = head.next

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    mid = slow.next
    slow.next = None

    # Sort both halves
    left = sortList(head)
    right = sortList(mid)

    # Merge
    return mergeTwoLists(left, right)
```

Complexity: $O(n \log n)$ time, $O(\log n)$ space for the recursion stack (no auxiliary array is allocated — the existing nodes are relinked in place by the merge step).

Recognition: "sort a linked list" → merge sort, because split-and-merge are both operations linked lists are naturally good at, unlike quicksort-style partitioning around random pivots.

Part 7 — More Classics

7.1 Copy list with random pointer

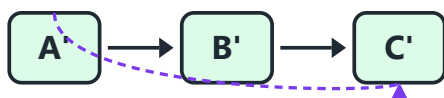
Each node now has a `next` **and** a `random` pointer, where `random` can point at *any* node in the list (or `None`) — not necessarily a nearby one.

Copy List with Random Pointer: old -> new HashMap

Original (random shown dashed):



Phase 1: old -> new copy (no links yet). Phase 2: `copy.next` / `copy.random` via the map.



`old_to_new.get(curr.random)` rebuilds the SAME shape on the copies.

The core difficulty: while copying node `A`, how do you know which *new* node corresponds to the *old* node that `A.random` points at, especially if that node hasn't been copied yet? The answer is a mapping — `original node -> copied node` — built in one pass, then consulted in a second pass:

```

def copyRandomList(head: Optional[RandomNode]) -> Optional[RandomNode]:
    if not head:
        return None

    old_to_new = {}

    # Phase 1: create copies
    curr = head

    while curr:
        old_to_new[curr] = RandomNode(curr.val)
        curr = curr.next

    # Phase 2: connect pointers
    curr = head

    while curr:
        copy = old_to_new[curr]

        copy.next = old_to_new.get(curr.next)
        copy.random = old_to_new.get(curr.random)

        curr = curr.next

    return old_to_new[head]

```

Phase 1 creates every copy (values only, no links yet) and records `old_to_new[old] = new`.

Phase 2 walks the original list again and, for each node, sets `copy.next` and `copy.random` by looking up the *originals'* `next` / `random` in the map. Because the map was fully populated in phase 1, every lookup in phase 2 succeeds regardless of whether the target node appears earlier or later in the list.

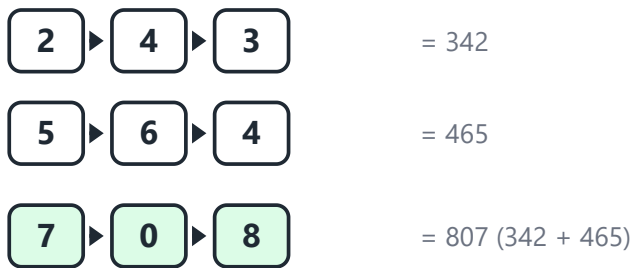
Complexity: $O(n)$ time, $O(n)$ space for the map.

Recognition: "clone a structure where pointers refer to arbitrary other nodes" → build an original-to-clone mapping first, then rewire using that mapping.

7.2 Add two numbers

Two numbers stored as linked lists, **least-significant digit first**: `2 -> 4 -> 3` represents 342. Add them and return the sum in the same representation.

Add Two Numbers: column addition, least-significant digit first



digit = (x + y + carry) % 10, carry = (x + y + carry) // 10; keep going while l1, l2, or carry remain.

This is just column addition, one digit position at a time, carrying into the next position exactly like adding numbers by hand:

```
def addTwoNumbers(l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
    dummy = ListNode(0)
    tail = dummy

    carry = 0

    while l1 or l2 or carry:
        x = l1.val if l1 else 0
        y = l2.val if l2 else 0

        total = x + y + carry

        carry = total // 10
        digit = total % 10

        tail.next = ListNode(digit)
        tail = tail.next

        if l1:
            l1 = l1.next

        if l2:
            l2 = l2.next

    return dummy.next
```

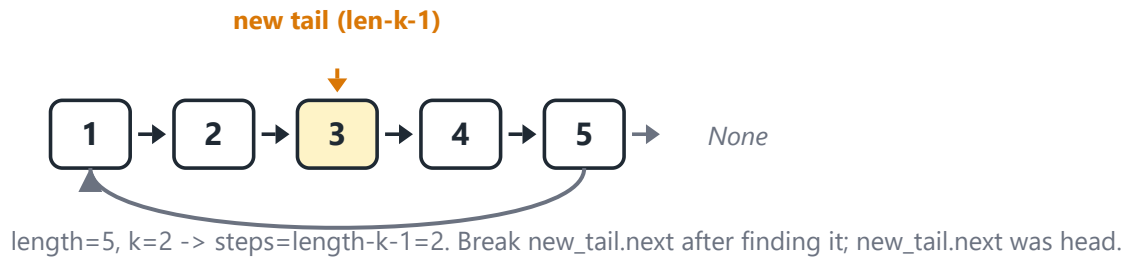
The loop condition `while l1 or l2 or carry` is worth noting carefully: it keeps going even after *both* input lists are exhausted, as long as there's a leftover carry (e.g. `5 + 5 = 10` needs one more digit than either input had).

Recognition: "numbers stored in reverse-order linked lists" → two pointers + running carry + dummy/tail, digit by digit.

7.3 Rotate list

Rotating `1 -> 2 -> 3 -> 4 -> 5` right by `k = 2` gives `4 -> 5 -> 1 -> 2 -> 3`.

Rotate List right by k: make circular, then break at the new tail



Rather than repeatedly moving the last node to the front ($O(nk)$), do it in one pass: find the length and the tail, connect the tail back to the head to form a temporary cycle, walk to what will become the new tail, and break the cycle there.

```

def rotateRight(head: Optional[ListNode], k: int) -> Optional[ListNode]:
    if not head or not head.next or k == 0:
        return head

    # Find length and tail
    length = 1
    tail = head

    while tail.next:
        tail = tail.next
        length += 1

    k %= length

    if k == 0:
        return head

    # Make circular
    tail.next = head

    # Find new tail
    steps = length - k - 1
    new_tail = head

    for _ in range(steps):
        new_tail = new_tail.next

    new_head = new_tail.next

    # Break circle
    new_tail.next = None

    return new_head

```

The `k %= length` step matters for correctness *and* efficiency: rotating by exactly `length` (or any multiple of it) returns the original list, and without the modulo a large `k` would make the "walk to new tail" loop needlessly (or incorrectly) long.

quantity	value
length	5
k % length	2
steps to new tail (length-k-1)	2

`rotateRight([1, 2, 3, 4, 5], k=2) -> [4, 5, 1, 2, 3]`

Recognition: "rotate a linked list" → make it circular → locate the new break point → break the cycle there.

Part 8 — Extension Problems

The source session's own closing offer was to *"continue with extra hard linked list problems and their solutions."* The six problems below take exactly that offer up — none of them were solved in the original session, but every one of them is a direct application of a pattern already established above. Working through them is a good test of whether you actually internalized the recognition rules or just followed along.

8.1 Partition list (around a value x)

Given a value `x`, rearrange the list so every node `< x` comes before every node `>= x`, **preserving the original relative order within each group**. This screams "two chains, then stitch them together" — a variant of the dummy + tail pattern from merging (Section 4.2), except here there's no comparison between two lists; every node from the *same* list is routed into one of two output chains based on a single test.

```
def partitionList(head: Optional[ListNode], x: int) -> Optional[ListNode]:
    """LC 86 – Partition List: stable-partition around x using two chains."""
    less_dummy = ListNode(0)
    more_dummy = ListNode(0)
    less_tail = less_dummy
    more_tail = more_dummy

    curr = head
    while curr:
        if curr.val < x:
            less_tail.next = curr
            less_tail = less_tail.next
        else:
            more_tail.next = curr
            more_tail = more_tail.next
        curr = curr.next

    more_tail.next = None
    less_tail.next = more_dummy.next

    return less_dummy.next
```

Build a `less` chain and a `more` chain with their own dummy heads and tails as you walk the original list once, then splice `more` onto the end of `less` and terminate it.

Recognition: "partition / group by a predicate while preserving order" → two dummy-headed chains, one pass, then concatenate.

8.2 Remove duplicates from a sorted list II

Unlike the simpler "keep one copy of each value" variant, this one must **delete every node whose value is duplicated anywhere**, keeping only values that appear exactly once. Because the list is sorted, all copies of a value are contiguous, so a single pass with a dummy node (for the case where the *very first* value is itself duplicated) suffices:

```
def deleteDuplicatesII(head: Optional[ListNode]) -> Optional[ListNode]:
    """LC 82 - Remove Duplicates from Sorted List II: drop ALL nodes that
    have a duplicate value, keeping only values that appear exactly once."""
    dummy = ListNode(0)
    dummy.next = head
    prev = dummy
    curr = head

    while curr:
        if curr.next and curr.next.val == curr.val:
            dup_val = curr.val
            while curr and curr.val == dup_val:
                curr = curr.next
            prev.next = curr
        else:
            prev = curr
            curr = curr.next

    return dummy.next
```

Whenever the current node's value equals its neighbor's, skip forward past the *entire run* of that value before reconnecting `prev.next`. **Recognition:** duplicates in a *sorted* list are always contiguous — the dummy + "skip the whole run" idea generalizes the ordinary "skip one node" deletion from Section 1.4.

8.3 Swap nodes in pairs

1 -> 2 -> 3 -> 4 becomes 2 -> 1 -> 4 -> 3. This is reverse-in-groups-of-k (Section 6.1) with `k` fixed at 2, small enough to write directly without the general group-finding machinery:

```
def swapPairs(head: Optional[ListNode]) -> Optional[ListNode]:
    """LC 24 – Swap Nodes in Pairs, using the dummy + local reversal pattern."""
    dummy = ListNode(0)
    dummy.next = head
    prev = dummy

    while prev.next and prev.next.next:
        first = prev.next
        second = first.next

        first.next = second.next
        second.next = first
        prev.next = second

        prev = first

    return dummy.next
```

A dummy node again removes the "is this the first pair?" special case; `prev` always points to the node just before the pair currently being swapped, exactly mirroring `group_prev` from the k-group reversal.

8.4 Reverse linked list II (reverse between positions)

Reverse only the sub-list from position `left` to `right` (1-indexed, inclusive), leaving everything outside that window untouched. This is *literally* Part 2's save/reverse/advance loop, just applied starting partway into the list instead of from `head`:

```

def reverseBetween(head: Optional[ListNode], left: int, right: int) ->
Optional[ListNode]:
    """LC 92 – Reverse Linked List II: reverse only the [left, right] window,
    reusing the exact save/reverse/advance loop from full reversal."""
    dummy = ListNode(0)
    dummy.next = head

    group_prev = dummy
    for _ in range(left - 1):
        group_prev = group_prev.next

    curr = group_prev.next
    prev = None
    for _ in range(right - left + 1):
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    # group_prev.next currently is the OLD start of the window (now the tail
    # of the reversed segment); it must connect to whatever curr is now.
    tail_of_reversed = group_prev.next
    tail_of_reversed.next = curr
    group_prev.next = prev

    return dummy.next

```

Walk `left - 1` steps to find the node just before the window (`group_prev`), then run the exact same reversal loop for `right - left + 1` iterations. The one detail that needs care: after the loop, `group_prev.next` 's **old** value (captured before the loop) is now the *tail* of the reversed segment and must be reconnected to `curr` (the first node past the window), while `group_prev.next` itself is repointed to `prev` (the new head of the reversed segment).

8.5 Odd-even linked list

Regroup nodes by their 1-indexed position parity — all odd-indexed nodes first (in original relative order), then all even-indexed nodes — **without allocating new nodes**:

```
def oddEvenList(head: Optional[ListNode]) -> Optional[ListNode]:
    """LC 328 – Odd Even Linked List: regroup by 1-indexed position parity."""
    if not head or not head.next:
        return head

    odd = head
    even = head.next
    even_head = even

    while even and even.next:
        odd.next = even.next
        odd = odd.next
        even.next = odd.next
        even = even.next

    odd.next = even_head

    return head
```

Maintain two running chains (`odd` , `even`) simultaneously by advancing both pointers two steps at a time through the original list, then splice `even` 's chain onto the end of `odd` 's.

Recognition: whenever a problem wants nodes regrouped by a parity/interleaving rule rather than by value, maintaining two parallel chains in a single pass is usually cheaper than two full separate passes.

8.6 Flatten a multilevel doubly linked list

Each node may have an extra `child` pointer to an independent doubly linked list, which itself may have further `child` lists nested inside it. Flatten the whole structure into a single doubly linked list, in depth-first order — whenever a node has a child, that child's entire (already-flattened) list gets spliced in immediately after it, before continuing to the node's original `next` .

```

def flatten(head: Optional[MultiNode]) -> Optional[MultiNode]:
    """LC 430 – Flatten a Multilevel Doubly Linked List (iterative, stack of
    "resume here" pointers – the same "save before you overwrite" rule that
    drove ordinary reversal now applies to the `next` we're about to detour
    away from)."""
    if not head:
        return None

    dummy = MultiNode(0, None, head, None)
    prev = dummy
    stack = [head]

    while stack:
        node = stack.pop()

        prev.next = node
        node.prev = prev

        if node.next:
            stack.append(node.next)

        if node.child:
            stack.append(node.child)
            node.child = None

        prev = node

    dummy.next.prev = None
    return dummy.next

```

The same "don't lose your only path to something you still need" rule from reversal applies here in a new guise: when you detour into `node.child`, you must **save** `node.next` **before you leave**, or you'll never find your way back to resume the current level. An explicit stack (pushing `node.next` before diving into `node.child`) makes this iterative and avoids Python's recursion-depth limit on deeply nested inputs. Note that `node.child` must be cleared to `None` on the way through — the problem requires the final structure to have no `child` pointers left, only `prev / next`.

Recognition: "nested/multilevel structure that needs flattening depth-first" → a stack holding "where to resume" pointers, in exactly the same spirit as saving `next_node` before rewriting `.next` during reversal.

Part 9 — Recognition, Templates & Decision Guide

9.1 Master pattern-recognition table

This is the table worth actually memorizing for interviews — not the code, this.

Problem wording	Pattern to think of
Traverse the list	<code>curr = curr.next</code>
Delete a node	predecessor → skip the node (<code>prev.next = curr.next</code>)
Reverse the list / a portion / groups of k	<code>prev</code> + <code>curr</code> + saved <code>next_node</code>
Find the middle	slow/fast (starting position of <code>fast</code> picks first vs. second middle)
Detect a cycle	slow/fast collision
Find the cycle's entry	collision → reset one pointer to <code>head</code> → walk together
Kth from the end	fixed gap of <code>k</code>
Remove kth from the end	dummy + gap <code>k + 1</code>
Merge sorted lists	two current pointers + dummy/tail
Reorder list	middle + reverse (2nd half) + interleave
Palindrome check	middle + reverse (2nd half) + compare
Intersection of two lists	swap heads at the end to equalize distance
O(1) removal of an arbitrary known node	doubly linked list
O(1) lookup and O(1) ordered reordering/removal	HashMap + Doubly Linked List (LRU)
Reverse every k nodes	locate group boundaries → reverse → reconnect → repeat
Merge k sorted lists	min-heap over the k current heads

Problem wording	Pattern to think of
Sort a linked list	merge sort (split via fast/slow, merge via the merge pattern)
Clone a structure with arbitrary internal pointers	original → clone HashMap
Numbers as reverse-order digit lists	two pointers + running carry + dummy
Rotate a list	make it circular → locate the new break → break it
Partition around a value	two dummy-headed chains, then concatenate
Remove all duplicated values (sorted)	dummy + skip the <i>entire</i> duplicate run
Flatten a nested/multilevel structure	stack of "resume here" pointers

9.2 The bigger mental framework

Rather than searching your memory for "which LeetCode solution is this," ask, in order:

1. **Do I just need to move in one direction?** → basic traversal, `curr = curr.next`.
2. **Do I need to modify links?** → think `prev` / `curr` / `next_node`, and save any link before you overwrite it.
3. **Do I need information from both ends at once?** → slow + fast, or reverse half the list and compare/interleave.
4. **Do I need kth-from-end information?** → a fixed gap between two pointers.
5. **Am I deleting something near the head?** → a dummy node.
6. **Am I combining sorted lists?** → two pointers + a tail pointer.
7. **Do I need O(1) lookup *and* ordered removal?** → HashMap + Doubly Linked List.
8. **Do I need to repeatedly reverse chunks?** → find boundaries → reverse → reconnect.
9. **Do multiple lists need merging at once?** → a min-heap.
10. **Does the problem look complicated?** → try decomposing it into patterns you already know, the way Reorder List turned out to be "middle + reverse + interleave" rather than one new algorithm.

9.3 The five pointer rules

1. Before changing a link, ask: "Do I still need the old `next`?"
2. If yes: save it first.

3. If deleting: think about the predecessor.
4. If the head creates a special case: consider a dummy node.
5. If the problem involves positions from both ends: think two pointers.

If you can consistently derive solutions from those five rules, you don't need to separately memorize twenty-plus linked-list solutions — you need to recognize which rule (or combination of rules) a new problem is asking for.

9.4 Condensed templates

Traversal

```
curr = head
while curr:
    # process curr
    curr = curr.next
```

Reversal (full list)

```
prev, curr = None, head
while curr:
    nxt = curr.next
    curr.next = prev
    prev, curr = curr, nxt
return prev
```

Fast/slow — second middle

```
slow = fast = head
while fast and fast.next:
    slow, fast = slow.next, fast.next.next
return slow
```

Cycle detection + entry

```

slow = fast = head
while fast and fast.next:
    slow, fast = slow.next, fast.next.next
    if slow is fast:
        slow = head
        while slow is not fast:
            slow, fast = slow.next, fast.next
        return slow # entry
return None # no cycle

```

Fixed gap (kth from end / remove kth from end)

```

dummy = ListNode(0, head) # include dummy only when DELETING
fast = dummy # (use head directly when just FINDING)
for _ in range(k): # k, or k+1 for deletion
    fast = fast.next
slow = dummy
while fast:
    slow, fast = slow.next, fast.next
# slow is now the target (find) or its predecessor (delete)

```

Merge two sorted lists

```

dummy = tail = ListNode(0)
while l1 and l2:
    if l1.val <= l2.val:
        tail.next, l1 = l1, l1.next
    else:
        tail.next, l2 = l2, l2.next
    tail = tail.next
tail.next = l1 or l2
return dummy.next

```

9.5 Decision guide

- **One pass, one direction, no structural change?** → plain traversal.
- **Structural change touching the head?** → dummy node, no exceptions.
- **Need a position relative to the *end*, or the *middle*?** → fast/slow with the right starting offset.
- **Need to know if you've "been here before"?** → fast/slow collision (cycle detection), not a visited-set, when O(1) space is required.
- **Two sorted inputs to combine?** → dummy + tail, compare-and-attach.
- **More than two sorted inputs?** → min-heap instead of chained pairwise merges.

- **Need to reverse *some* window, not the whole list?** → the same reversal loop, just start/stop it at the right positions (Section 8.4 is the general case).
- **Need $O(1)$ access *and* $O(1)$ reordering by an arbitrary key?** → HashMap + Doubly Linked List.
- **Structure nested inside itself (multilevel)?** → an explicit stack of "resume here" pointers, not naive recursion if depth is a concern.

Part 10 — Interview Q&A Review

Work through these the way the original session did: answer before reading the explanation.

Q: `curr = curr.next` just executed. Did the list change? No. Only the `curr` variable's reference moved. No node's `.next` field was written. This is the single most load-bearing distinction in the whole topic (Section 1.3).

Q: Why does `hasCycle` check `while fast and fast.next`, not just `while fast`? If `fast` is on the last node, `fast.next` is `None`; evaluating `fast.next.next` afterward would crash. Checking both guarantees the two-step advance is always safe (Section 3.1–3.3).

Q: Two pointers `slow` and `fast` just collided inside a cycle. Is the collision node the cycle's entry point? Not necessarily. It's *some* node inside the cycle, but generally not the entry. Finding the entry needs the second phase: reset one pointer to `head`, then advance both one step at a time until they meet again (Section 3.4).

Q: Why does resetting one pointer to `head` and walking both one step at a time land on the cycle entry? Because $\text{distance}(\text{head} \rightarrow \text{entry}) \equiv \text{distance}(\text{meeting} \rightarrow \text{entry}) \pmod{\text{cycle length}}$ — not exactly equal, congruent modulo the cycle length. Walking the same number of steps from congruent starting distances lands both pointers at the same position around the cycle (the full derivation is in Section 3.4, including the correction made mid-derivation in the source session).

Q: You need the 2nd-from-last node of a list. Do you need the list's length first? No — create a gap of 2 between two pointers (advance one pointer 2 steps ahead of the other), then move both together until the leading one falls off the end. The trailing pointer lands on the target in one pass (Section 3.5).

Q: You need to *delete* the 2nd-from-last node instead of just finding it. Does the gap stay at 2? No, it becomes $k + 1 = 3$. Deletion needs the *predecessor* of the target, not the target itself, so the gap has to be one larger (Section 4.1).

Q: `prev.next = curr.next` deletes `curr` — but what if `curr` is the head? There's no `prev` for the head, so that line doesn't apply; you'd need `head = head.next` as a special case instead. A dummy node placed before `head` removes the special case entirely by giving the head an artificial predecessor (Section 1.4–1.5).

Q: Reorder List looks like a single complicated algorithm. Is it? No — it decomposes into three patterns you already have: find the middle, reverse the second half, then interleave the two halves. Recognizing the decomposition is the actual interview skill being tested, not inventing a new algorithm (Section 4.3).

Q: Why can `mergeTwoLists` just do `tail.next = list1 or list2` once one list is exhausted, instead of continuing to compare? Because both lists are individually sorted. Whatever remains in the non-exhausted list is already sorted internally *and* already larger than everything already placed — no further comparison can change the outcome (Section 4.2).

Q: Two linked lists might intersect. Why does swapping heads at the end make two unequal-length pointers meet exactly at the intersection? Because it equalizes total distance traveled: one pointer covers *(its own prefix) + (shared part) + (other list's prefix)*, and so does the other, just in the opposite order. Same total distance means they arrive at the shared portion simultaneously — or both hit `None` simultaneously if there's no intersection (Section 5.1).

Q: Why does an LRU cache need *both* a hashmap and a doubly linked list — wouldn't one of them alone work? A hashmap alone gives $O(1)$ lookup but no ordering; you couldn't cheaply find "the least recently used" entry. A linked list alone gives ordering but $O(n)$ lookup by key. Together, the hashmap finds the node in $O(1)$ and the doubly linked list moves/removes it in $O(1)$ once found (Section 5.3).

Q: Merging 2 sorted lists uses two pointers and a comparison. Does merging k lists just extend that to k pointers and k-way comparisons? That works but is inefficient if done naively (comparing all k candidates every step). A min-heap holding each list's current head gives the same result with each step costing $O(\log k)$ instead of $O(k)$ (Section 6.2).

Q: `sortList` recurses by splitting at the middle and merging. What's the time and space complexity, and why can't you just convert to an array, sort, and rebuild? $O(n \log n)$ time, $O(\log n)$ space for the recursion (not counting output). Converting to an array and back would work but isn't in the spirit of the problem (and typically isn't allowed in an interview framing that asks for $O(1)$ *extra* space beyond recursion) — merge sort is the structure-respecting approach because both splitting and merging are things linked lists do natively (Section 6.3).

Q: Copy List with Random Pointer — why do you need two full passes instead of one? Because `random` can point at a node that hasn't been copied yet when you reach it in a single forward pass. Building the complete `old -> new` mapping first (pass 1), *then* wiring up every `.next / .random` using that mapping (pass 2), guarantees every lookup in pass 2 succeeds regardless of ordering (Section 7.1).

Q: Add Two Numbers loops `while l1 or l2 or carry`. Why include `carry` in that condition? Because a carry can outlive both input lists — e.g. `5 + 5 = 10` needs a new final digit even after both single-digit inputs are exhausted (Section 7.2).

Q: Rotate List makes the list temporarily circular. Why is that safe, and why must `k %= length` happen first? It's safe because the circularity is broken again before returning — the caller never observes it. `k %= length` matters because rotating by a multiple of the list's length returns the original list, and skipping the modulo would make the "find the new tail" walk unnecessarily (or, for `k` larger than length in a naive off-by-one implementation, incorrectly) long (Section 7.3).